

Research Procedure: LLM-Based Android Bridge Security Analysis

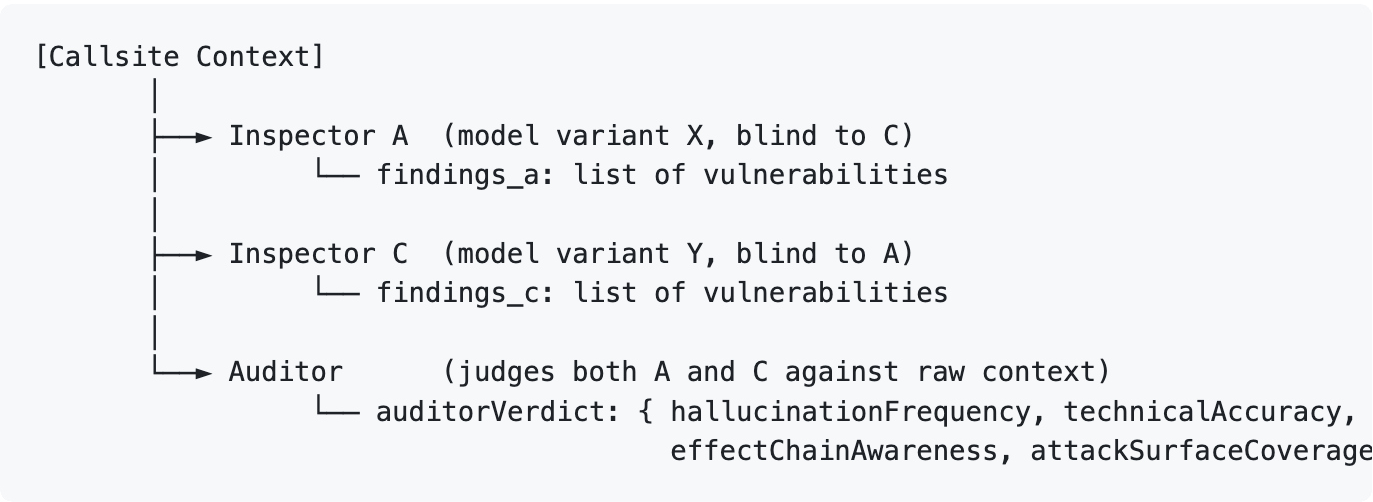
This document is the authoritative step-by-step guide for researchers running experiments with this tool. It covers environment setup, pipeline execution, Gold Standard construction, and data analysis. Follow all phases in order on a clean run.

Table of Contents

- 1. [Conceptual Overview](#)
- 2. [Environment Setup](#)
- 3. [Configuring a Run](#)
- 4. [Running the Pipeline](#)
- 5. [Understanding the Output](#)
- 6. [Building the Gold Standard](#)
- 7. [Inter-Rater Reliability](#)
- 8. [Data Analysis](#)
- 9. [Reporting Checklist](#)
- 10. [Known Issues and Gotchas](#)

1. Conceptual Overview

The pipeline uses three roles per callsite:



A **callsite** is one registration of `addJavascriptInterface` in a single Android app — a specific Java bridge interface bound to a WebView at a specific code location. Each

callsite produces one record in `callsite_results.json`.

The **researcher** is you. The `researcherConclusion` block in each record is your manually assigned ground truth. It is the only thing that makes the auditor's scores scientifically valid.

2. Environment Setup

2.1 Python dependencies

```
cd analyzer/  
python -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

2.2 API keys (cloud models)

Create `analyzer/.env`:

```
GEMINI_API_KEY=your_key_here
```

Leave unused keys absent rather than blank — a blank string causes authentication errors.

2.3 Local models (Ollama)

Install [Ollama](#) and pull any models you intend to use:

```
ollama pull llama3.2:3b  
ollama pull gemma3:4b  
ollama pull gemma3:12b  
# add others as needed
```

Verify they are available:

```
ollama list
```

2.4 Database

The pipeline reads from `analyzer/data/Intent.sqlite`, produced by [iWanDroid](#). Ensure this file exists before running.

3. Configuring a Run

All model selection is driven by `analyzer/analysis_configuration.json` . Edit this file before each experimental run.

```
{
  "auditor": {
    "provider": "ollama",
    "model": "gemma3:12b"
  },
  "inspector_pairs": [
    {
      "inspector_a": { "provider": "ollama", "model": "llama3.2:3b" },
      "inspector_c": { "provider": "ollama", "model": "gemma3:4b" }
    }
  ]
}
```

Supported providers: `ollama` , `gemini` .

Multiple pairs: Add more objects to `inspector_pairs` to test additional model combinations in a single run. Each pair produces one entry in the `permutations` array of every callsite record.

Auditor model choice: The auditor should be the strongest available model for your run, ideally one that was not used as an inspector. A weaker auditor will produce noisy verdicts that obscure real differences between inspectors.

Record what you used: Before running, copy the config into your lab notebook or a results subdirectory. The config is not stored automatically alongside results.

4. Running the Pipeline

From inside the `analyzer/` directory:

```
cd analyzer/
python src/main.py
```

Optional flags:

Flag	Effect
<code>--config PATH</code>	Use an alternative config file

<code>--limit N</code>	Analyze only the first N apps (useful for pilot runs)
------------------------	---

For a quick sanity check before a full run:

```
python src/main.py --limit 1
```

The pipeline prints progress to stdout. Results are written incrementally to `analyzer/results/callsite_results.json` — if the run is interrupted, partial results are preserved.

5. Understanding the Output

`callsite_results.json` is a JSON array. Each element has this shape:

```
{
  "identifiedCallsiteID": "Intent-webview_new-4",
  "context": "APP: ...\n\nBRIDGE:\n...\n\nJS SNIPPETS:\n...",
  "permutations": [
    {
      "researcher": 0,
      "inspectors": ["ollama/llama3.2:3b", "ollama/gemma3:4b"],
      "auditor": "ollama/gemma3:12b",
      "results": [
        {
          "inspector": "ollama/llama3.2:3b",
          "findings": ["### Finding: ...", "..."]
        },
        {
          "inspector": "ollama/gemma3:4b",
          "findings": ["### Finding: ...", "..."]
        }
      ],
      "auditorVerdict": {
        "ollama/llama3.2:3b": {
          "hallucinationFrequency": 0.4,
          "technicalAccuracy": 7,
          "effectChainAwareness": 6,
          "attackSurfaceCoverage": 0.6
        },
        "ollama/gemma3:4b": { "..." }
      },
      "researcherConclusion": {
        "hallucinationFrequency": 0.0,
        "technicalAccuracy": 0,
        "effectChainAwareness": 0,
        "attackSurfaceCoverage": 0.0
      }
    }
  ]
}
```



```
]
}
```

Field reference

Field	Type	Description
<code>identifiedCallsiteID</code>	string	Stable ID. Format: <code><db_stem>-<seq></code> . All records sharing a prefix come from the same app.
<code>context</code>	string	Raw evidence given to every model. Bridge interface (Java methods) + up to 10 JS snippets.
<code>permutations[].inspectors</code>	string[]	The two inspector model IDs used in this permutation.
<code>permutations[].results[i].findings</code>	string[]	Each element is one <code>### Finding:</code> block from that inspector.
<code>permutations[].auditorVerdict</code>	object	Automated scores keyed by inspector model ID. See metric definitions below.
<code>permutations[].researcherConclusion</code>	object	Ground truth. Manually filled by researcher. All zeros until you annotate it.

Metric definitions

Metric	Range	Meaning
<code>hallucinationFrequency</code>	0.0 – 1.0	Fraction of findings that reference methods, classes, or behaviors absent from the context.
<code>technicalAccuracy</code>	1 – 10	Correctness and precision of the non-hallucinated findings.
<code>effectChainAwareness</code>	1 – 10	Quality of reasoning about chained consequences, not just identifying a dangerous method.
<code>attackSurfaceCoverage</code>	0.0 – 1.0	Fraction of real vulnerabilities present in the context that were identified.

The `auditorVerdict` contains the automated LLM estimate of these metrics. The

The `auditorVerdict` contains the automated LLM estimate of these metrics. The `researcherConclusion` contains your manual estimate of the same metrics for the same permutation, aggregated across both inspectors.

6. Building the Gold Standard

The Gold Standard is the set of researcher-assigned `researcherConclusion` values. Without it, the auditor verdicts cannot be validated.

6.1 What you are annotating

For each **permutation** of a callsite, you assign one `researcherConclusion` object. This is a holistic judgment about the overall quality of that permutation's output (both inspectors together), not per-inspector.

If a run has multiple `inspector_pairs`, each pair produces its own `permutations` entry and requires its own annotation.

6.2 Annotation protocol

Open `callsite_results.json` and work through records sequentially. For each record:

Step 1 — Read the context. Read `context` carefully. List every bridge method that actually exists. Note whether JS snippets are present. This is your ground truth. What you see here is all the model knew.

Step 2 — Identify the real attack surface. Before looking at the findings, write down which methods you consider genuinely risky and why. This prevents confirmation bias from the model's output influencing your judgment. You need this list to compute `attackSurfaceCoverage`.

Step 3 — Evaluate each finding. For each finding string in `results[i].findings`, ask:

- Does the method or object cited in **Evidence** exist in the `context`? If not → hallucination.
- Is the data flow path ([JS Source] → [Bridge Object] → [Java Sink]) traceable step by step in the context? If invented → hallucination.
- Is the technical description consistent with what the method signature actually does?
- Does the finding reason about consequences, or just assert danger?

Step 4 — Fill in `researcherConclusion`.

Compute each metric as follows:

hallucinationFrequency Count total findings across both inspectors. Count how many are hallucinated (reference absent methods or behaviors). Divide: `hallucinated / total`. If both inspectors produced zero findings, set to 0.0.

total . If both inspectors produced zero findings, set to `0.0` .

Example: Inspector A has 4 findings (1 hallucinated), Inspector C has 5 findings (2 hallucinated) → $3 / 9 \approx 0.33$.

technicalAccuracy (integer 1–10) Considering only the non-hallucinated findings, how technically sound are they on average? Use this rubric:

Score	Meaning
9–10	Findings are precise, the data flow path is fully traceable, severity is proportionate.
7–8	Mostly sound; minor imprecision in path description or severity inflation/deflation.
5–6	Correct risk category but vague or mixed-up evidence.
3–4	Several findings are in the right domain but factually wrong in key details.
1–2	Almost no technical basis; findings are generic and not grounded in the context.

effectChainAwareness (integer 1–10) How well do the inspectors reason about what happens *after* a method is called? Generic "this is dangerous" scores low. A finding that traces `sendToChat(String)` → `inflateUi()` → consumer execution → privilege escalation scenario scores high.

Score	Meaning
9–10	Multi-step chains articulated; worst-case impact clearly described.
7–8	Impact described at one level beyond the immediate method call.
5–6	Impact mentioned but not reasoned through.
3–4	Only the sink is named; no impact described.
1–2	No impact reasoning present.

attackSurfaceCoverage (float 0.0–1.0) From your Step 2 list of real vulnerabilities, what fraction did the combined output of both inspectors identify? `identified / total_real` . If your Step 2 list is empty (no real attack surface), set to `1.0` only if both inspectors correctly reported no findings; otherwise `0.0` .

6.3 Callsites with empty JS snippets

Many callsites have no JS snippets (the field is blank after `JS SNIPPETS:`). In these cases:

- Any finding that depends on JS-side behavior (e.g., "the JS transmits PII to localStorage")

is automatically hallucinated — there is no JS evidence.

- `attackSurfaceCoverage` must be assessed solely from the Java bridge methods.
- Inspectors cannot be penalized for not identifying JS-side issues when no JS was provided.

6.4 Annotation format

Edit `callsite_results.json` in place. Replace the zeros in `researcherConclusion` :

```
"researcherConclusion": {  
  "hallucinationFrequency": 0.33,  
  "technicalAccuracy": 6,  
  "effectChainAwareness": 5,  
  "attackSurfaceCoverage": 0.5  
}
```

Keep a separate annotation log (spreadsheet or markdown file) with one row per callsite containing your reasoning. This is essential for inter-rater reliability and for writing the methods section of a paper.

6.5 Minimum sample size

Annotating every callsite is ideal but may be impractical. A statistically meaningful subset requires:

- At minimum **30 callsites** for pilot correlation checks.
- At least **60 callsites** for the Wilcoxon signed-rank test comparing model architectures (RQ B).
- Sample across app types: include callsites both with and without JS snippets, and callsites from different bridge interface names.

Prioritize variety over volume. A stratified random sample is better than annotating the first N records.

7. Inter-Rater Reliability

If two or more researchers annotate independently, measure agreement before merging.

7.1 Continuous metrics

For `hallucinationFrequency` and `attackSurfaceCoverage` (continuous), use **intraclass correlation coefficient (ICC)** — specifically ICC(2,1) for absolute agreement with two or more raters. Report the 95% confidence interval.

Target: ICC ≥ 0.75 before merging. Below this, discuss disagreements and re-annotate.

7.2 Ordinal metrics

For `technicalAccuracy` and `effectChainAwareness` (1–10 integer scales), use **weighted Cohen's kappa** with quadratic weights, or Spearman's ρ between rater vectors.

Target: $\kappa \geq 0.60$ (substantial agreement).

7.3 Resolving disagreements

For any callsite where raters disagree by more than 2 points on ordinal metrics, or by more than 0.2 on continuous metrics:

1. Both raters independently write out their reasoning.
2. Discuss and identify the specific point of disagreement (e.g., "is this a hallucination if the method exists but the vulnerability claim is wrong?").
3. Apply the result to the annotation protocol to prevent recurrence.
4. Adjudicate by consensus. Do not average without discussion.

8. Data Analysis

After annotation, load `callsite_results.json` into a Python notebook. The analyses below map to the three research questions.

8.1 Data loading and normalization

```
import json, pandas as pd

with open("analyzer/results/callsite_results.json") as f:
    records = json.load(f)

rows = []
for rec in records:
    cid = rec["identifiedCallsiteID"]
    for perm in rec["permutations"]:
        for model_id, verdict in perm["auditorVerdict"].items():
            rows.append({
                "callsiteID": cid,
                "inspector": model_id,
                "auditor": perm["auditor"],
                # Normalize: coerce strings to numbers (known pipeline bug)
                "aud_hallucinationFrequency": float(verdict["hallucinationFrequency"]),
                "aud_technicalAccuracy": int(verdict["technicalAccuracy"]),
                "aud_effectChainAwareness": int(verdict["effectChainAwareness"]),
                "aud_attackSurfaceCoverage": float(verdict["attackSurfaceCoverage"])
```

```

        # Researcher labels (aggregated per permutation, not per inspector
        "res_hallucinationFrequency": float(perm["researcherConclusion"]["
        "res_technicalAccuracy": int(perm["researcherConclusion"]["technic
        "res_effectChainAwareness": int(perm["researcherConclusion"]["effe
        "res_attackSurfaceCoverage": float(perm["researcherConclusion"]["a
    })

```

```
df = pd.DataFrame(rows)
```

Note the type coercion on `hallucinationFrequency` and `technicalAccuracy` — the pipeline sometimes emits these as strings. Always normalize before analysis.

8.2 RQ A — What combination of metrics best captures Trustworthiness?

Goal: Determine whether a composite Trustworthiness Score derived from the auditor's four metrics correlates with the researcher's ground truth labels. If it does, the auditor is a valid proxy for human judgment.

Step 1 — Compute a candidate Trustworthiness Score.

A simple composite:

```

df["trustworthiness"] = (
    (df["aud_technicalAccuracy"] / 10) * 0.4 +
    (df["aud_effectChainAwareness"] / 10) * 0.3 +
    df["aud_attackSurfaceCoverage"] * 0.3 -
    df["aud_hallucinationFrequency"] * 0.5
).clip(-1, 1)

```

The weights (0.4 / 0.3 / 0.3) and hallucination penalty (0.5) are initial hypotheses. You will vary these in Step 3.

Step 2 — Compute a researcher reference score using the same formula applied to `res_*` columns.

Step 3 — Measure correlation.

```

from scipy.stats import spearmanr, pearsonr

rho, p = spearmanr(df["trustworthiness"], df["res_trustworthiness"])
print(f"Spearman ρ = {rho:.3f}, p = {p:.4f}")

```

Use Spearman (not Pearson) because the ordinal metrics are not normally distributed.

Step 4 — Vary the formula. Systematically adjust weights and the hallucination penalty. Record (ρ, p) for each variant. The combination that maximizes ρ while keeping $p < 0.05$

is your best-fit Trustworthiness Score. Report all variants tested — do not cherry-pick.

Step 5 — Validate on a held-out set. If your annotated sample is large enough (≥ 60 callsites), split 70/30: fit weights on the training set, report ρ on the held-out set only.

Expected output: A table of weight combinations and their Spearman ρ values. The winning combination becomes the definition of Trustworthiness Score used in RQ B and RQ C.

8.3 RQ B — How do model architectures compare?

Goal: Given the Trustworthiness Score from RQ A, determine whether different inspector model architectures produce significantly different scores on the same callsites.

Prerequisite: You must have run the pipeline with at least two different `inspector_pairs` using different model architectures (e.g., one pair using `llama3.2:3b` vs `gemma3:4b`, another using a GPT or larger Gemini model).

Step 1 — Pair the scores by callsite.

Each callsite has one `researcherConclusion` (the ground truth) and potentially multiple auditor verdicts from different model combinations. Align by `callsiteID`.

```
# Example: pivot so each row is a callsite, columns are per-model trustworthiness
pivot = df.pivot_table(
    index="callsiteID",
    columns="inspector",
    values="trustworthiness"
).dropna() # keep only callsites where all models ran
```

Step 2 — Paired statistical test.

Because each callsite is analyzed by all models, use the **Wilcoxon signed-rank test** (nonparametric paired test):

```
from scipy.stats import wilcoxon

stat, p = wilcoxon(pivot["ollama/llama3.2:3b"], pivot["ollama/gemma3:4b"])
print(f"Wilcoxon W = {stat}, p = {p:.4f}")
```

Report effect size using rank-biserial correlation:

```
n = len(pivot)
r = 1 - (2 * stat) / (n * (n + 1) / 2)
print(f"Effect size r = {r:.3f}")
```

Step 3 — Report per-metric breakdowns.

The composite score may hide which dimension drives the difference. Run the Wilcoxon test separately for each of the four raw metrics. This is important for RQ C (failure taxonomy) and helps distinguish "model B hallucinates less" from "model B has better effect chain reasoning".

Step 4 — Compare auditor verdicts against researcher labels.

For each model, compute how often the auditor's judgment agreed with yours:

```
df["halluc_agreement"] = abs(df["aud_hallucinationFrequency"] - df["res_hallucinat
```

A high agreement rate on hallucination detection means the auditor is reliable for that model pairing. Low agreement is a finding in itself.

Expected output: A table of models × metrics with Wilcoxon p-values and effect sizes. A bar chart of mean Trustworthiness Score per model architecture. Statement of whether the null hypothesis (no difference between architectures) can be rejected.

8.4 RQ C — Where does the Auditor struggle or shine?

Goal: Identify failure modes — types of callsites or vulnerability classes where the auditor's verdict diverges most from the researcher's ground truth.

Step 1 — Compute per-callsite auditor error.

```
df["halluc_error"] = df["aud_hallucinationFrequency"] - df["res_hallucinationFreque  
df["coverage_error"] = df["aud_attackSurfaceCoverage"] - df["res_attackSurfaceCove
```

Positive `halluc_error` means the auditor over-estimated hallucinations (false alarms). Negative means it missed real hallucinations.

Step 2 — Segment by context richness.

Tag each callsite as `has_js` (JS snippets present) or `no_js`:

```
df["has_js"] = df["callsiteID"].map(  
    lambda cid: len(next(r for r in records if r["identifiedCallsiteID"] == cid))["  
)
```

Compare mean errors between groups. The hypothesis is that auditor errors are higher when JS snippets are absent — the model has less grounding.

Step 3 — Segment by bridge complexity.

Count the number of bridge methods per callsite (count occurrences of `[METHOD]` in the context string). Compute Spearman correlation between method count and auditor error. The hypothesis is that more complex bridges (larger context) lead to higher auditor error — this directly tests the context window / cross-language reasoning failure mode.

Step 4 — Failure taxonomy.

For callsites where `|halluc_error| > 0.3` (substantial auditor error), read the findings manually and classify the failure:

Category	Description
Method confusion	The auditor accepted a finding about a method that doesn't exist in the bridge.
Severity inflation	The auditor rated a speculative finding as technically accurate.
Cross-language gap	A valid data flow spanning JS and Java was missed because the auditor failed to connect both sides.
Context overflow	The bridge has many methods; the auditor appeared to confuse them or lose track.
Correct rejection	The auditor correctly identified a hallucination the researcher also flagged.

Build a frequency table of these categories. This is your qualitative failure taxonomy.

Expected output: Boxplots of auditor error segmented by `has_js` . Scatter plot of method count vs. auditor error with p and p-value. A frequency table of the failure taxonomy categories with representative examples quoted from the findings.

9. Reporting Checklist

Before writing up results, verify:

- ☐ `researcherConclusion` filled for all annotated callsites — no remaining zeros in the analysis set.
- ☐ Inter-rater ICC and kappa computed and reported (if multiple annotators).
- ☐ All `auditorVerdict` numeric fields normalized to correct types (no string-encoded numbers).
- ☐ Model configuration for each run recorded (provider, model name, auditor model).

- ☐ Wilcoxon tests use only callsites where all compared models ran (no missing data in the pivot).
 - ☐ Effect sizes reported alongside p-values.
 - ☐ All formula variants tested for RQ A reported — not only the best-fit.
 - ☐ Failure taxonomy based on manual reading, not automated classification.
 - ☐ Callsites with empty JS snippets handled consistently (noted in methods section).
-

10. Known Issues and Gotchas

Type inconsistency in `auditorVerdict`: The pipeline sometimes emits numeric metrics as JSON strings (`"7"` instead of `7`). Always coerce to the correct type before computing. The data loading snippet in Section 8.1 handles this.

`researcherConclusion` is a permutation-level aggregate: It represents your judgment about the combined output of both inspectors in that permutation, not one inspector. When computing `hallucinationFrequency` for the researcher label, count findings across both inspectors together.

Empty findings list: An inspector that returned `findings: []` scores `attackSurfaceCoverage = 0.0` for that inspector's contribution. It does not mean the callsite has no attack surface.

`researcherConclusion.researcher` field: The `researcher` integer field in each permutation is a placeholder for future multi-researcher workflows. It is always `0` currently. Do not use it for grouping.

Re-running the pipeline overwrites results: The output file is reset to `[]` at the start of each run. Back up `callsite_results.json` and your annotations before re-running with a different config.

Auditor hallucination metric is relative, not absolute: The auditor's `hallucinationFrequency` is normalized: the inspector with more hallucinations gets `1.0`, the other gets `their_count / max_count`. This means two inspectors that both hallucinate equally will both receive a moderate score rather than both getting `1.0`. Account for this when comparing auditor vs researcher labels which use raw fractions